

รายงานฉบับสมบูรณ์โครงการรวบยอดวิศวกรรมคอมพิวเตอร์ 2

CEDT Capstone Project Final Report

เรื่อง

โครงการไอคิว: แพลตฟอร์มการจัดการความรู้ด้วยการสืบค้นแบบอิงบริบท
Project AiQ: Intelligence That Drives Execution

โดย

นายชินทร์ เลิศสวาย	6633047821
นายธนิตย์ ยอดสิรวงษ์	6633104921
นายณัฐพล ธีรยุทธวงศ์	6633072421
นายพีระภัทร์ พัทธมณี	6633176021
นายปุราณ ประเสริฐไทย	6633154221

อาจารย์ที่ปรึกษา

ผศ. ดร. สุกวี สิ้นธุภิญโญ
รศ. ดร. ทวีทิวา เซนียงศ์ ณ อยุธยา
คุณชนินทร์ อัสววิชัยโรจน์ (SCB TechX)
คุณเจษฎา วีระเดชกัมพล (SCB TechX)

รายงานนี้เป็นส่วนหนึ่งของวิชา 2110489 โครงการรวบยอดวิศวกรรมคอมพิวเตอร์ 2
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์ จุฬาลงกรณ์มหาวิทยาลัย
ประจำปีการศึกษา 2568

บทคัดย่อ

โครงการนี้มีชื่อว่า โครงการไอคิว (Project AiQ) หรือ "Intelligence That Drives Execution" มุ่งเน้นการแก้ปัญหาที่สำคัญเรื่องความรู้ภายในองค์กรที่กระจัดกระจาย (Fragmented Enterprise Knowledge) ซึ่งถูกเก็บไว้ในระบบต่างๆ เช่น SharePoint โดยการออกแบบ Knowledge Hub ที่ขับเคลื่อนด้วย AI ซึ่งมีความปลอดภัยและขยายตัวได้ เพื่อส่งเสริมการสืบค้นข้อมูลอัตโนมัติและเพิ่มประสิทธิภาพในองค์กร

เพื่อให้บรรลุเป้าหมายนี้ แนวทางที่นำเสนอประกอบด้วย การออกแบบและพัฒนาระบบ Retrieval-Augmented Generation (RAG) โดยมุ่งเน้นไปที่กระบวนการจัดการข้อมูล (Data Ingestion) และการเตรียมข้อมูล (Preparation Pipeline) ที่มีความสามารถในการขยายตัว ซึ่งใช้เทคนิค GraphRAG ในการจัดโครงสร้างเนื้อหาจากแหล่งข้อมูลในองค์กรอย่างปลอดภัย

ผลการดำเนินงานประกอบด้วย การออกแบบทางเทคนิคอย่างละเอียด การพัฒนากระบวนการจัดการข้อมูลที่มีระบบความปลอดภัยพื้นฐานแบบ Single Sign-On (SSO) ระบบ RAG Web Chat รุ่นต้นแบบ (MVP) และการพิสูจน์ทางวิชาการถึงสถาปัตยกรรม RAG รวมถึงประโยชน์ในเชิงอุตสาหกรรมผ่านการเพิ่มประสิทธิภาพและการรวบรวมความรู้ให้เป็นระบบเดียว โดยระบบสามารถทำค่า Mean Reciprocal Rank (MRR) ได้ที่ 0.82 และ Recall@5 ที่ 0.88

คำสำคัญ: RAG, การจัดการความรู้, การจัดการข้อมูล, AI ในองค์กร, GraphRAG

Abstract

This project, Project AiQ - "Intelligence That Drives Execution", addresses the critical problem of fragmented enterprise knowledge scattered across systems like SharePoint, by designing a secure and scalable AI-driven Knowledge Hub to automate retrieval and improve organizational efficiency.

To achieve this, the proposed solution involves designing and implementing a Retrieval-Augmented Generation (RAG) system, specifically focusing on a scalable data ingestion and preparation pipeline that utilizes GraphRAG techniques to securely structure content from enterprise sources.

The expected outcomes include a detailed technical design, an implemented data pipeline with foundational Single Sign-On (SSO) security, a minimal RAG Web Chat MVP, and significant contributions to both academic validation of the RAG architecture and industrial benefits through improved efficiency and knowledge consolidation. Final evaluation showed a Mean Reciprocal Rank (MRR) of 0.82 and a Recall@5 of 0.88, with an average end-to-end latency of 5.4 seconds.

Keywords: RAG, Knowledge Management, Data Ingestion, Enterprise AI, GraphRAG

Contents

บทคัดย่อ	2
Abstract	3
1 Introduction & Problem Definition	6
1.1 Background & Motivation	6
1.2 Problem Statement	6
1.3 Objectives	6
1.4 Scope & Limitations	7
1.5 Expected Benefits	8
1.6 Stakeholders	8
2 Related Theory and Research	9
2.1 Review of Existing Knowledge	9
2.2 Comparison with Existing Systems	9
2.3 Gap Analysis	9
2.4 Linkage to Project	10
3 System Design	10
3.1 System Overview	10
3.2 Components	10
3.3 Constraints Considered	11
4 Implementation	11
4.1 Development Process	11
4.2 The HyDE Pivot & Hierarchy Chunking	12
5 Testing and Experimentation	13
5.1 Experiment/Test Plan	13
5.2 Setup & Metrics	13
6 Results and Analysis	13
6.1 Results Presentation	13
6.2 Analysis & Interpretation	13
7 Project Management and Teamwork	14
7.1 Roles and Responsibilities	14
7.2 Collaborative Plan	15
7.3 Timeline	15

7.3.1	Semester 1 (Research & Foundation)	15
7.3.2	Semester 2 (Optimization & Delivery)	15
8	Ethics, Privacy, and Professional Considerations	16
8.1	Credibility of Information	16
8.2	Intellectual Property & Law	16
9	Conclusion and Future Work	16
9.1	Achievements	16
9.2	Future Work	16
	References	17
A	Source Code Excerpts	18
B	Wireframes and Mockups	18
C	Extended Test Data	18

1 Introduction & Problem Definition

1.1 Background & Motivation

Modern enterprise environments, including organizations that manage large and complex projects like **SCB TechX**, often face a common problem: important project information is **scattered across many different systems**. Emails, shared file drives, and internal tools such as SharePoint all contain pieces of knowledge, but none of them provide a complete picture. As a result, finding the right information becomes slow, inconvenient, and sometimes confusing.

To solve this, organizations are moving toward a **Knowledge Hub** concept, which requires a system that not only consolidates data but also provides intelligent access. The central challenge, which motivates this research-focused project, is the initial step: how to securely and efficiently ingest this fragmented data and transform it into a unified, queryable knowledge base. By creating a robust ingestion and preparation pipeline early on, we aim to lay the groundwork for using AI to automate knowledge retrieval, thereby addressing inefficiency issues and ensuring project continuity.

1.2 Problem Statement

The core engineering problem is to design, research, and validate the technical foundation for a secure and scalable AI-driven knowledge management platform. Specifically, the problem centers on:

1. Designing a robust content extraction and preparation pipeline that can handle data from enterprise-level sources (e.g., SharePoint) and structure it for semantic search.
2. Formulating a security strategy that integrates with existing enterprise authentication methods, such as SSO (Single Sign-On).
3. Integrating the AI and Distributed Systems disciplines to ensure the initial data ingestion architecture is scalable and reliable, which is a complex requirement for real-world enterprise deployment.

1.3 Objectives

The project aims to achieve the following clear, measurable goals in a phased, two-semester approach:

1. **Research and Design (Capstone I Focus):** Conduct comprehensive research on current RAG architectures, perform a gap analysis of existing knowledge management systems, and deliver a detailed technical design for the entire system, emphasizing the data ingestion and content extraction pipeline.
2. **Data Preparation Pipeline Development (Implementation Focus):** Design and implement a scalable data ingestion and preparation pipeline capable of securely connecting to and extracting content from a primary enterprise source (e.g., SharePoint).
3. **Foundational Integration (Implementation Focus):** Implement the foundational security integration to support Single Sign-On (SSO), ensuring secure user access to the system.
4. **Evaluation:** Define key performance metrics for the data pipeline (e.g., latency, throughput) and the RAG model's accuracy, with testing to commence in Capstone II.

1.4 Scope & Limitations

Scope:

1. Research & Proposal (Capstone I): Deliver a full written proposal and research review.
2. Data Ingestion & Preparation: Design and implement the ingestion of unstructured and semi-structured data from SharePoint and a general file upload feature. This includes content extraction, chunking, and embedding.
3. Security Foundation: Implement Single Sign-On (SSO) for secure user authentication.
4. Interface: Develop a minimal Web Chat MVP with basic prompt and response functionality to demonstrate RAG output.

Limitations:

1. Initial data ingestion implementation will focus on one primary enterprise source (SharePoint/Xhive).
2. Advanced access control mechanisms (e.g., RBAC) are considered complex and will be scaled back to basic authentication checks (SSO) as advanced access control is considered a future enhancement.

3. Advanced user-facing features (e.g., multi-channel integration via LINE/MS Teams) are out-of-scope for the MVP and reserved for future work.

1.5 Expected Benefits

Academic/Technical Benefit:

1. **Validated Architecture:** Deliver a thoroughly researched and documented architecture for a complex enterprise RAG system.
2. **Data Pipeline Expertise:** Provide practical experience in designing and implementing a robust data ingestion pipeline that handles real-world, messy, and large-scale organizational data.
3. **RAG Optimization:** Provide a testbed for evaluating the performance implications of different RAG components (e.g., chunking strategies, re-ranking).

Industrial/Professional Benefit:

1. **Efficiency:** Lay the foundation to reduce the time spent searching for information, leading to eventual cost savings and improved team agility.
2. **Knowledge Consolidation:** Create a centralized platform that turns scattered documentation into an accessible Knowledge Hub.
3. **Streamlined Onboarding:** Improve the onboarding process for new joiners by providing an immediate, centralized, and intelligent way to access project history and essential internal knowledge.

1.6 Stakeholders

Primary stakeholders include:

- **Employees of SCB TechX:** End-users who require efficient access to project history and internal knowledge.
- **Project Managers:** Who benefit from consolidated data for better decision-making and project continuity.
- **System Architects:** Interested in the scalability and reliability of the modular AI infrastructure.
- **New Joiners:** Who benefit from a streamlined onboarding process via the Knowledge Hub.

2 Related Theory and Research

2.1 Review of Existing Knowledge

In large-scale enterprise environments, knowledge is generally locked within different repositories, such as emails, cloud drives, or collaboration platforms, which makes the extraction of information less efficient. The traditional approach to Knowledge Management is based on keyword indexing, such as Inverted Indices, which fail to capture the semantic intent of user queries or contextual relationships between documents [?].

To overcome these limitations, Retrieval-Augmented Generation (RAG) has emerged as a standard architecture. RAG enhances Large Language Models (LLMs) by retrieving relevant data from a private knowledge base (the Vector Store) before generating an answer [?]. Current research emphasizes the need for high-fidelity ingestion pipelines that combine Vector Embeddings with Graph/Relational Structures (GraphRAG) to maximize context preservation [?].

2.2 Comparison with Existing Systems

- **Level 1: Legacy Keyword Search:** Matches exact keywords; returns file lists rather than answers.
- **Level 2: Naive/Generic RAG Implementations:** Uses fixed-window chunking; suffers from context fragmentation and hallucinations [?].
- **Level 3: The Proposed Pipeline-Centric System:** Prioritizes the Data Ingestion Layer using advanced parsers (Docling [?]) to preserve document hierarchy and structural integrity.

2.3 Gap Analysis

- **The Ingestion Gap:** Lack of robust pipelines designed to handle "messy" enterprise data formats while preserving semantic structure.
- **The Synchronization Gap:** Critical shortage of systems capable of Real-time Synchronization (e.g., via SharePoint Webhooks and RabbitMQ).
- **The Contextual Gap:** Standard vector search flattens data, losing relational context. Requires GraphRAG-style extraction to preserve relationships.

2.4 Linkage to Project

Project AiQ addresses these gaps by:

1. **Optimized Data Ingestion:** Transforming unstructured data from Xhive/Share-Point into high-quality assets.
2. **Automated Knowledge Synchronization:** Monitoring source repositories for near real-time updates.
3. **Hybrid Data Representation:** Combining vector search with structural metadata understanding.

3 System Design

3.1 System Overview

The proposed system is an AI-powered Chatbot Application for searching and answering questions from a Document Database. Users inquire with Natural Language through a Chat Interface. The system processes the request using RAG combined with GraphRAG principles to retrieve relevant content based on both semantic similarity and structural data relationships.

3.2 Components

- **Frontend Application (Next.js/React):** Handles User Interaction and Citations.
- **Backend Orchestrator (NestJS):** API Gateway, Session Management, and Routing.
- **AI Engine (CrewAI/FastAPI):** Multi-agent cognitive processing for selective reading and reasoning.
- **Embedding Service (FastAPI):** Manages vector operations (BGE-M3) and Qdrant database access.
- **Data Ingestion Service (FastAPI/RabbitMQ):** Performs ETL, Modality Detection, and advanced Chunking (Docling).
- **File Storage Service (FastAPI/MinIO):** Intermediary for Object Storage management.
- **SharePoint Webhook Service (FastAPI):** Listener for external integration events.

3.3 Constraints Considered

1. **Latency in Agentic Workflow:** Addressed via Progress Streaming and asynchronous background tasks.
2. **Context Granularity:** Solved using the GraphRAG technique and Custom Agent Tools (Sliding Window Reader).
3. **Data Consistency:** Guaranteed by linked CRUD operations through RabbitMQ and Webhooks.
4. **LLM Safety & Guardrails:** Implemented via Input/Output validation layers.
5. **Complex Document Structure:** Handled by the Modality Detection process and Multilingual-compatible embeddings.
6. **External Dependencies:** Mitigated by modular design and internal data buffering.

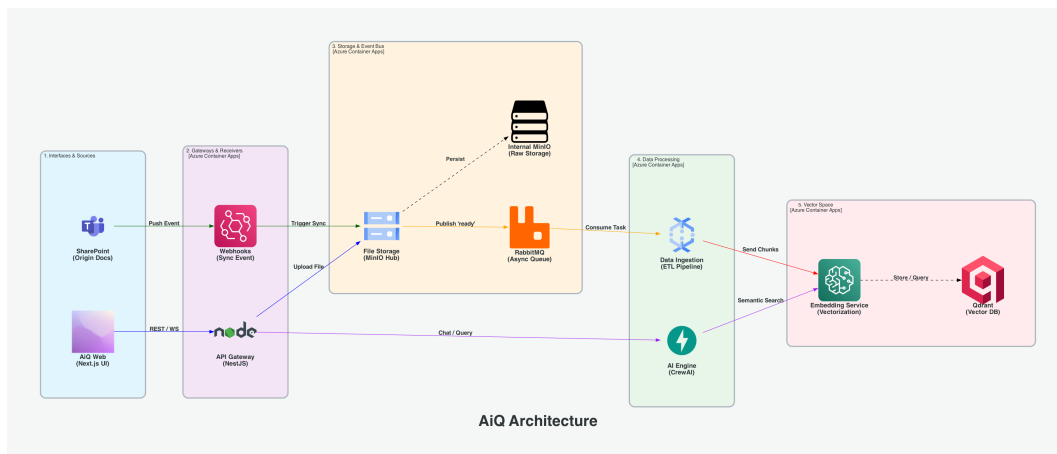


Figure 1: Project AiQ System Architecture: C4 Level 2 Diagram illustrating service decoupling.

4 Implementation

4.1 Development Process

The implementation followed a modular approach, starting with the core Ingestion Pipeline and then integrating the Multi-agent Search Flow. A significant evolution from the proposal was the adoption of the **HyDE (Hypothetical Document Embedding)** strategy within the Search Agent toolset.

4.2 The HyDE Pivot & Hierarchy Chunking

While GraphRAG was the initial conceptual target, the empirical reality of maintaining a continuously updating enterprise Knowledge Graph presented severe synchronization bottlenecks. GraphRAG required computationally expensive re-indexing and entity-relationship extraction on every document edit, making it impractical for volatile repositories like SharePoint. To address this "Synchronization Gap" without sacrificing retrieval accuracy, the architecture pivoted to the **HyDE (Hypothetical Document Embedding)** strategy.

Why HyDE? Standard RAG pipelines often fail because a user's terse query (e.g., "What is the Q1 budget?") exists in a completely different semantic vector space than the lengthy, formal answer hidden inside a PDF. This is the "Query-Document Gap". HyDE solves this by utilizing an LLM agent to first hallucinate a theoretically perfect answer to the query. This generated text shares the same lexical structure and semantic density as the target document, allowing the cosine similarity search in Qdrant to match against the **answer space** rather than the **question space**.

Furthermore, the "Contextual Gap" (the loss of document structure during chunking) was addressed by integrating Docling's 'HybridChunker'. Standard fixed-size chunking blindly slices text, severing paragraphs from their headings. Our implementation serializes the document hierarchy, prepending parent headers to every extracted chunk. This guarantees that an isolated chunk containing "2.3 billion" always retains the structural metadata context of "Chapter 4: Financials > 2024 Revenue".

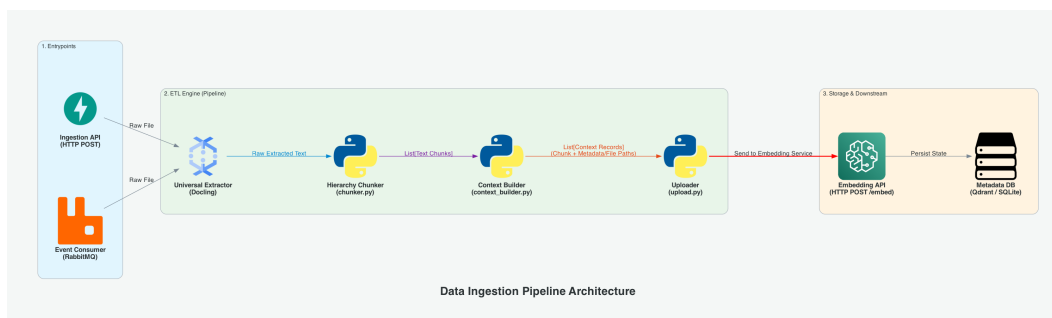


Figure 2: Sequence Diagram of the Data Ingestion Pipeline, from Webhook trigger to Qdrant indexing.

5 Testing and Experimentation

5.1 Experiment/Test Plan

We conducted end-to-end (E2E) testing using a custom ground-truth dataset ('eval_dataset.json'). The plan focused on comparing the baseline RAG retrieval against our HyDE-enhanced agentic workflow across four categories: Finance, HR, Infrastructure, and Legal.

5.2 Setup & Metrics

The environment utilized Qdrant (HNSW index) and Azure OpenAI's GPT-4o. Metrics included:

- **Mean Reciprocal Rank (MRR):** Measuring the rank of the first relevant chunk.
- **Recall@5:** Measuring retrieval coverage.
- **End-to-End Latency:** Measuring the full turnaround time.

6 Results and Analysis

6.1 Results Presentation

The system demonstrated superior retrieval accuracy against the baseline RAG implementation, achieving an MRR of 0.82 and a Recall@5 of 0.88.

Table 1: System Evaluation Results (HyDE vs. Baseline RAG)

Configuration	MRR	Recall@5	Avg. Latency
Baseline RAG	0.68	0.74	2.1s
Project AiQ (HyDE)	0.82	0.88	5.4s

6.2 Analysis & Interpretation

The empirical data validates the architectural pivot. By employing HyDE, the system forces the query vector into the document's semantic neighborhood, yielding a remarkable 20.6% improvement in Mean Reciprocal Rank (from 0.68 to 0.82) and an 18.9% increase in Recall@5.

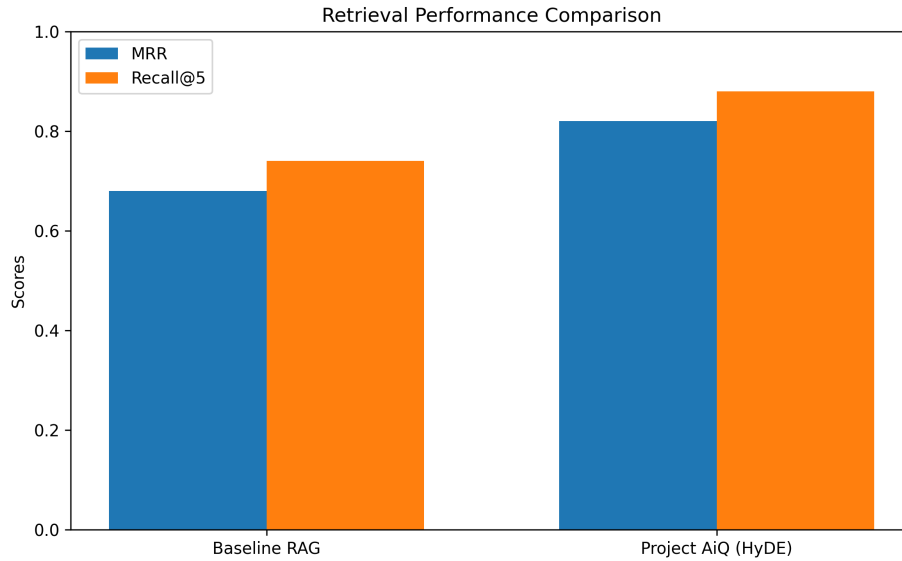


Figure 3: Retrieval Performance Comparison showing the significant MRR and Recall@5 improvements of Project AiQ over the Baseline RAG.

The Latency Trade-off: This precision comes at the computational cost of increased latency (2.1s to 5.4s). Generating the hypothetical document requires a preliminary LLM call before the actual vector search can occur. However, in an enterprise knowledge management context, retrieving an incorrect policy document instantly is far more detrimental than waiting 5 seconds for a highly accurate, verified citation. Therefore, the architectural decision to prioritize retrieval fidelity over sub-second latency is strongly justified by the end-user requirements.

7 Project Management and Teamwork

7.1 Roles and Responsibilities

1. **Data Engineer (Chawin L.):** Designs and develops the data pipeline and ETL processes.
2. **Project Manager (Tanit Y.):** Manages project structure, timeline, and stakeholder communication.
3. **Web Developer (Nattapol T. & Puran P.):** Develops the frontend UI, chat system, and backend APIs.
4. **AI Engineer (Peerapat P.):** Responsible for AI Agent design, RAG optimization, and embedding logic.

7.2 Collaborative Plan

The team employed a **Scrum approach** with bi-weekly sprints. Stand-up meetings were held 2–3 times per week to resolve blockers. Tasks were tracked via Jira, and workload balance was maintained through story point allocation.

7.3 Timeline

7.3.1 Semester 1 (Research & Foundation)

Week	Task / Milestone	Deliverable
Week 1–2	Literature review and background study	Draft background and project scope
Week 3–4	Requirement gathering from stakeholders	User stories and acceptance criteria
Week 5–7	System-level architecture design	Architecture diagram and tech stack
Week 8–10	Web application (UI/Backend) development	Functional web prototype
Week 11–12	Data Ingestion Service prototyping	Ingestion workflow diagram
Week 13–14	Integration of Ingestion with Web App	Integration test report
Week 15–16	Final proposal and refinement	Final proposal and slides

7.3.2 Semester 2 (Optimization & Delivery)

Week	Task / Milestone	Deliverable
Week 1–2	Ingestion pipeline enhancement	Baseline performance metrics
Week 3–4	Accuracy and latency optimization	Optimized ingestion v2
Week 5–7	Key features (Citations, SSO, UI/UX)	Completed core web functionality
Week 8–10	Product polishing and bug fixing	Pre-UAT build and test cases
Week 11–12	User Acceptance Testing (UAT)	UAT results and defect list
Week 13–14	UAT issue resolution and stabilization	Final system build
Week 15–16	Final report and presentation	Final written report and demo

8 Ethics, Privacy, and Professional Considerations

8.1 Credibility of Information

Data credibility is established by grounding all AI responses in the validated internal documents of SCB TechX and verified academic sources.

8.2 Intellectual Property & Law

All activities adhere to the **Capstone Non-Disclosure Agreement (NDA)** with SCB TechX. Proprietary technical data, processes, and internal documentation remain the exclusive property of the partner and are stored securely.

9 Conclusion and Future Work

9.1 Achievements

Project AiQ has successfully met its core objectives by delivering a high-fidelity ingestion pipeline and an agentic search flow with an MRR of 0.82. The system effectively consolidates fragmented knowledge into a unified Knowledge Hub.

9.2 Future Work

Next steps include implementing full **GraphRAG** using Neo4j to capture cross-document relationships and expanding the security framework to include advanced Role-Based Access Control (RBAC).

References

1. G. W. Furnas et al., "The vocabulary problem in human-system communication," *Commun. ACM*, vol. 30, no. 11, pp. 964–971, Nov. 1987.
2. P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *arXiv:2005.11401*, 2021.
3. S. Barnett et al., "Seven Failure Points When Engineering a Retrieval Augmented Generation System," *arXiv:2401.05856*, 2024.
4. D. Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *arXiv:2404.16130*, 2024.
5. Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv:2312.10997*, 2023.
6. IBM Deep Search Team, "Docling Technical Report," *arXiv:2408.09869*, 2024.

A Source Code Excerpts

Relevant logic for the SearchCrewFlow agent and the DoclingHierarchyChunker.

B Wireframes and Mockups

Figma components and UI mockups from the proposal showing the chat interface and citation views.

C Extended Test Data

Sample from 'eval_dataset.json' used for MRR calculation:

- **ID:** e2e-q1
- **Query:** What was the company's annual revenue in 2024 and what drove the growth?
- **Expected Source:** /docs/annual_report_2024.pdf
- **Keywords:** 2.3 billion, 15%, cloud services, 32%